

Resumen teórico — Sistemas Operativos

Notas del curso: *Usuarios y seguridad*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo *Usuarios y seguridad* de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se presentan los mecanismos que un SO debe ofrecer para soportar múltiples usuarios: la identificación mediante usuarios y grupos (con sus bases de datos en `/etc/passwd`, `/etc/shadow` y `/etc/group`), el proceso de login y la autenticación basada en contraseñas con *hash criptográfico* y *salt* (incluyendo PAM y la autenticación multifactor), el control de acceso a recursos mediante la *matriz de control de acceso* y su implementación como ACLs (con el modelo UNIX U/G/O de 9 bits y las *full ACLs*), los modelos DAC y MAC (con un ejemplo de Bell-LaPadula), los privilegios de un proceso (real UID, effective UID, `setuid`, `su`, `sudo`) y las *capacidades* como mecanismo más fino de control de privilegios. El objetivo es servir de material de estudio para el final de la materia.

Índice

1. Introducción	2
2. Usuarios, grupos y archivos	2
2.1. Grupos y recursos	2
2.2. El usuario privilegiado	3
2.3. Gestión de usuarios y grupos	3
2.4. Bases de datos en UNIX	3
3. Login y autenticación	4
3.1. Formato de <code>/etc/shadow</code>	4
3.2. Hash criptográfico	4
3.3. Verificación en el login	4
3.4. Salt y ataques por diccionario	4
3.5. Otros mecanismos de autenticación	5
3.6. PAM (Pluggable Authentication Modules)	5
4. Control de acceso	5
4.1. Matriz de control de acceso	5
5. Listas de control de acceso (ACLs)	6
5.1. El modelo UNIX: U/G/O	6
5.2. Comandos UNIX	6
5.3. Limitación y full-ACLs	6
5.4. DAC vs. MAC	6
6. Privilegios de un proceso	7
6.1. Effective UID y <code>setuid</code>	7
6.2. Otros mecanismos: <code>su</code> y <code>sudo</code>	7

7. Capacidades	8
7.1. Qué son las capacidades	8
7.2. Capacidades como reemplazo de <code>setuid-root</code>	8
8. Cierre	8

1. Introducción

Soportar **múltiples usuarios** en un sistema operativo no es sólo una cuestión de comodidad: implica un conjunto de mecanismos que el SO debe definir e implementar para que los usuarios convivan sin pisarse, sin invadirse y sin agotar los recursos compartidos. Las Notas listan siete responsabilidades centrales:

1. **Separación de ambientes de trabajo** de cada usuario en el sistema de archivos (cada usuario tiene su propio *home*, sus archivos privados, sus configuraciones).
2. **Identificar y autenticar** a los usuarios.
3. Relacionar programas, procesos y usuarios (todo programa que se ejecuta corre “a nombre de” algún usuario).
4. Establecer **límites** en la utilización de recursos para que todos los usuarios puedan trabajar.
5. Ofrecer **servicios** (comandos y syscalls) de gestión de usuarios.
6. Ofrecer mecanismos de **control de acceso** a recursos.
7. Opcionalmente, ofrecer múltiples terminales/consolas (reales o virtuales) para que varios usuarios puedan operar en paralelo.

Idea intuitiva

A grandes rasgos, los temas que siguen tratan: *quién* es un usuario (sección 2), *cómo* demuestra que es quien dice ser (sección 3), *a qué* puede acceder (secciones 4 y 5) y con *qué privilegios* corre un proceso suyo (secciones 6 y 7).

2. Usuarios, grupos y archivos

Un SO asocia a cada usuario con:

- un **identificador único** (en UNIX, un entero positivo, el *uid*),
- un **rótulo** o *nombre de usuario*, y
- un conjunto de **credenciales de autenticación**.

2.1. Grupos y recursos

Definición

Un **grupo** representa un conjunto de usuarios. Se usa para facilitar la gestión del control de acceso: en lugar de listar permisos usuario por usuario, basta con asignar permisos al grupo y agregar al usuario al grupo.

Un **recurso** es un archivo de datos o programa, un mecanismo de IPC (por ejemplo, un pipe), un proceso o un dispositivo de hardware. El SO debe proteger los recursos respecto a quién puede acceder a cada uno.

2.2. El usuario privilegiado

Importante

En todo sistema existe *al menos un* **usuario privilegiado**. Es quien puede crear nuevos usuarios y asignarles roles o permisos. No tiene restricciones de acceso a los recursos y actúa como **administrador** del sistema.

- En UNIX se llama **root**.
- En MS-Windows se llama **Administrador**.

2.3. Gestión de usuarios y grupos

El sistema ofrece utilidades para crear/eliminar usuarios y grupos. En UNIX:

```
adduser alice # crear usuario
deluser alice # eliminar usuario
addgroup devs # crear grupo
delgroup devs # eliminar grupo
```

2.4. Bases de datos en UNIX

En los sistemas tipo UNIX, los datos de los usuarios viven en dos archivos del directorio `/etc`:

`/etc/passwd`. Contiene registros (uno por usuario) de la forma:

```
user:x:uid:gid:full name:homedir:shell
```

- **user**: nombre de usuario.
- **x**: si está presente, significa que la contraseña está en `/etc/shadow` (separación deliberada por seguridad).
- **uid**, **gid**: identificadores numéricos de usuario y grupo principal.
- **full name**: nombre completo (campo descriptivo).
- **homedir**: directorio *home*.
- **shell**: shell que se le abre al hacer login (por ejemplo `/bin/bash`).

Usuarios sin login. Una configuración típica incluye usuarios que *no* tienen acceso interactivo: usan `/usr/bin/nologin` como shell. Ejemplos: `bin`, `sys`, `mail`. Permiten asociar un *dueño*, un grupo y permisos a programas o servicios del sistema sin que nadie pueda iniciar sesión con esa cuenta.

Identificadores especiales.

- `root` tiene `uid = 0`.
- Al primer usuario *real* del sistema se le asigna comúnmente `uid = 1000`.

`/etc/group`. Cada usuario tiene un grupo *principal* y puede ser miembro de otros. La base de grupos está en `/etc/group`.

Idea intuitiva

La idea de poner la contraseña en `/etc/shadow` (legible sólo por `root`) y dejar `/etc/passwd` legible por todo el mundo es lo que permite a comandos comunes (`ls -l`, `ps`, ...) traducir uids a nombres sin exponer los hashes de las contraseñas.

3. Login y autenticación

El sistema ejecuta en cada terminal el programa `login`, que generalmente pide el nombre de usuario y las credenciales de autenticación. El método más común es la presentación de una **contraseña** (*password*).

- Cada usuario puede cambiar su contraseña con `passwd` (UNIX).
- El administrador asigna la contraseña inicial al crear el usuario.

3.1. Formato de `/etc/shadow`

Las contraseñas se almacenan en `/etc/shadow`. Cada línea tiene el formato:

```
user:hashed-password:last-changed:min-days:max-days:warn-days:ina...
```

- `user`: nombre del usuario.
- `hashed-password`: **hash de la contraseña**, comúnmente prefijado por `$n$`, donde n indica el código del algoritmo de hash utilizado.
- Los demás campos contienen fechas o días desde el último cambio, período en el que el sistema debería pedir el cambio de contraseña, días de inactividad y la fecha de expiración de la cuenta.

3.2. Hash criptográfico

Para no almacenar el secreto en forma plana, se guarda $hash(password)$. La función hash es un **hash criptográfico** (MD5, SHA, ...).

Definición

Una función **hash criptográfica** es una función $hash : \{0, 1\}^* \rightarrow \{0, 1\}^n$, donde n es el número de bits de la salida. Toma una entrada de longitud arbitraria y produce una salida de longitud fija. Por ejemplo, MD5 produce 128 bits.

Propiedad clave. Una de las propiedades deseables es la **resistencia a preimágenes**: dado el par $(x, hash(x))$, debe ser *muy* difícil encontrar un x' tal que $hash(x') = hash(x)$. Esto requiere gran poder de cómputo por ser un problema NP-Hard.

Importante

Actualmente esa propiedad está *quebrada* para MD5: es relativamente fácil romperla con el cómputo moderno, por lo que MD5 no debe usarse para autenticación de contraseñas.

3.3. Verificación en el login

El programa `login` autentica al usuario verificando que:

$$hash(provided_password) = saved_password.$$

Es decir, nunca compara contraseñas en texto plano; sólo hashes.

3.4. Salt y ataques por diccionario

Comúnmente, `passwd` agrega al password provisto un prefijo llamado **salt**: un valor generado aleatoriamente (en UNIX, típicamente dos caracteres) que se almacena como prefijo del hash.

Por qué hace falta. El salt previene los **ataques por diccionario**.

Idea intuitiva

Un ataque por diccionario aplica cuando un atacante *roba* el archivo de contraseñas. El atacante dispone de un *diccionario* generado previamente: un conjunto de pares (*plaintext*, *hash(plaintext)*) para millones de palabras y combinaciones comunes. Si una contraseña (hash) robada coincide con el segundo elemento de algún par del diccionario, el atacante ya conoce el texto plano correspondiente: la inferencia se reduce a una búsqueda en el diccionario.

El **salt** hace que para cada usuario se calcule el hash con un prefijo distinto. Para que el ataque sirva, el diccionario debería contener *todas* las combinaciones (*plaintext* + *salt*); generarlo se vuelve incomparablemente más costoso.

3.5. Otros mecanismos de autenticación

El sistema de login puede aceptar otras credenciales:

- **Biométricos** en sistemas físicos: huellas dactilares, reconocimiento de rostro, etc.
- **Autenticación multifactor**. Por ejemplo, doble factor: el usuario provee, además, un código enviado por un servicio de mensajería (mail, chat, ...) o generado por un generador de códigos de autenticación.

3.6. PAM (Pluggable Authentication Modules)

GNU/Linux ofrece un sistema de autenticación muy flexible llamado **Pluggable Authentication Modules (PAM)**. PAM permite *configurar*, *usar* y *desarrollar* mecanismos de autenticación que pueden combinarse (por ejemplo: contraseña + huella + token).

4. Control de acceso

El sistema debe asociar **permisos de acceso** (lectura, escritura o ejecución) entre los usuarios y los recursos (archivos de datos y programas).

4.1. Matriz de control de acceso

Conceptualmente, esto se modela mediante una **matriz de control de acceso**: las filas son los recursos, las columnas son los usuarios, y cada celda contiene los permisos que el usuario tiene sobre el recurso. Por ejemplo:

resources / users	f1	f2	...
root	rwX	rw-	...
...	...	...	...
alice	r--	---	...

Importante

Implementar esta matriz directamente es un desafío: podría ser *inmensa* por la cantidad de recursos y usuarios (millones de archivos por miles de usuarios). Por eso los SO la representan de forma comprimida, comúnmente por *columnas* (ACLs) o por *filas* (*capability lists*).

5. Listas de control de acceso (ACLs)

Una **ACL** (*Access Control List*) es una representación de la matriz **por columnas**: a cada recurso se le asocia el conjunto $\{(user, perms), \dots\}$ de pares usuario-permiso.

5.1. El modelo UNIX: U/G/O

Para representar este conjunto de forma compacta, en UNIX los usuarios se clasifican en tres clases:

1. **U (user)**: el dueño (*owner*) del archivo.
2. **G (group)**: el grupo (principal) asociado al archivo.
3. **O (others)**: el resto de los usuarios del sistema.

Los permisos se asignan al recurso *para cada clase*: permisos del dueño, permisos del grupo y permisos del resto.

Representación de 9 bits. Sólo hacen falta 9 bits: tres para *rwX* del dueño, tres para el grupo y tres para los otros:

$$\underbrace{rwx}_U \quad \underbrace{r--}_G \quad \underbrace{r--}_O$$

Estos permisos forman parte de los **metadatos** de cada archivo (viven en su *inode*).

5.2. Comandos UNIX

```
chown alice file # cambiar dueño
chgrp devs file # cambiar grupo
chmod 644 file # cambiar permisos (rw-r--r--)
```

5.3. Limitación y full-ACLs

Idea intuitiva

La agrupación en sólo tres clases a veces *no* provee la flexibilidad necesaria: por ejemplo, dar lectura a Alice y Bob pero no a otros miembros del grupo no es expresable con 9 bits.

Los SO modernos ofrecen **full-ACLs**, que permiten asociar el conjunto de pares (usuario, permisos) de manera *explícita*. Se implementan comúnmente como archivos especiales (ocultos) asociados a cada archivo. En GNU/Linux:

```
getfacl file # listar ACL extendida
setfacl -m u:alice:rw file # otorgar rw a alice
```

5.4. DAC vs. MAC

Definición

Un sistema en el que los usuarios pueden modificar los permisos de sus recursos (de los cuales son dueños) se denomina **discretionary access control (DAC)**. UNIX por defecto es DAC: el dueño decide quién puede leer/escribir/ejecutar su archivo.

Existen modelos en los que los permisos son asignados por una **autoridad central de seguridad**, no por el dueño. Se conocen como **mandatory access control (MAC)** y son comunes en organizaciones militares.

Ejemplo: Bell-LaPadula. Asigna a los usuarios un *nivel de seguridad* y a los recursos un *rótulo*. Existe un orden parcial (retículo) de niveles:

$$TOP\ SECRET > SECRET > CONFIDENTIAL > PUBLIC.$$

Un general con nivel *SECRET* podrá acceder a recursos con rótulo *SECRET*, *CONFIDENTIAL* o *PUBLIC*, pero *no* podrá acceder a recursos *TOP SECRET*.

6. Privilegios de un proceso

Un proceso usa los permisos de su **usuario real** (*real UID*): el usuario que lanzó su ejecución. Esto le permite acceder a los recursos permitidos a ese usuario.

6.1. Effective UID y `setuid`

Definición

El **effective UID** (*eid*) de un proceso corresponde al usuario cuyos permisos efectivamente usa el kernel para validar accesos. En general coincide con el real UID. Pero, si un programa tiene activado el permiso especial **setuid**, al ejecutarse el *eid* del proceso pasa a ser el del *dueño del programa*, no el del usuario que lo invocó.

Caso típico: `passwd`. El programa `/usr/bin/passwd` tiene dueño `root` y requiere acceso a `/etc/shadow`, el cual sólo es legible/escrible por `root` (para evitar que cualquier usuario pueda copiarlo y atacarlo). Pero cualquier usuario debe poder cambiar su propia contraseña.

- `passwd` tiene activado el bit **setuid**.
- Cuando Alice lo ejecuta, el proceso adquiere los permisos de `root`.
- De ese modo, el proceso puede actualizar `/etc/shadow` a pesar de haber sido lanzado por Alice.

Importante

Esto es un mecanismo de **escalado de privilegios**: pasar temporalmente a un nivel de privilegios mayor para realizar una operación concreta. Es poderoso pero peligroso: cualquier bug en un programa `setuid-root` puede ser explotado para obtener `root` completo.

6.2. Otros mecanismos: `su` y `sudo`

Comandos que permiten cambios de privilegio bajo control:

- **`su`:** permite crear una nueva sesión (*shell*) con otro usuario (por omisión, `root`). `su` solicita las credenciales de autenticación del *otro* usuario; éste debe tener permitido el login.
- **`sudo`:** permite ejecutar un comando como otro usuario. Sólo requiere reingresar las credenciales del *usuario actual*. Su configuración vive en `/etc/sudoers` y especifica qué usuarios pueden ejecutar qué comandos bajo los permisos de qué otros usuarios.

Idea intuitiva

La diferencia clave es de quién se piden las credenciales: **su** exige conocer la contraseña del usuario destino (típicamente root), mientras que **sudo** confía en el usuario actual previa configuración en `/etc/sudoers`. Por eso **sudo** se prefiere en sistemas multi-administrador: no hay que compartir la contraseña de root.

En MS-Windows el comando equivalente a **su** es **runas**.

7. Capacidades

Que un proceso adquiera privilegios de **root** (*todos* los permisos) puede generar problemas de seguridad: requiere que el programa no contenga errores y que acceda *sólo* a los recursos estrictamente necesarios.

7.1. Qué son las capacidades

Definición

Una **capacidad** (*capability*) define qué operaciones se pueden aplicar sobre un recurso. Comúnmente se usan para *limitar* el conjunto de operaciones posibles, en lugar de otorgar privilegios totales.

Por ejemplo, si se abre un archivo en modo de sólo lectura, el descriptor contiene sólo la capacidad de lectura, inhibiendo cualquier operación de escritura o truncado.

7.2. Capacidades como reemplazo de setuid-root

Muchos SO modernos usan capacidades en programas privilegiados para limitar el conjunto de operaciones o recursos a los que pueden acceder.

Ejemplo: tcpdump. El programa **tcpdump** permite *capturar* paquetes que pasan por una interfaz de red. Para esto requeriría privilegios de **root**. La solución moderna es aplicar una **capacidad** que sólo permita acceder a *raw sockets*: en GNU/Linux esa capacidad es **CAP_NET_RAW**.

Idea intuitiva

En lugar de darle a **tcpdump** “poder absoluto” como setuid-root, se le otorga *exactamente* la capacidad que necesita. Si el binario tuviera un bug explotable, el atacante ganaría **CAP_NET_RAW**, no root completo: un daño mucho menor.

Para más información sobre capacidades en GNU/Linux:

```
man capabilities
```

8. Cierre

El subsistema de usuarios y seguridad implementa, en conjunto, la idea de que un sistema multi-usuario debe *identificar* a sus usuarios, *autenticar* su identidad y *autorizar* cada operación que intentan realizar. Las ideas principales:

- Cada usuario tiene un **uid** y pertenece a uno o más **grupos**. La separación de `/etc/passwd`, `/etc/shadow` y `/etc/group` es la concreción de este modelo en UNIX. Root (`uid = 0`) es el único usuario sin restricciones.

- La **autenticación** compara hashes, no texto plano. Una función hash criptográfica garantiza que conocer $hash(x)$ no permite recuperar x con facilidad. El **salt** es la defensa específica contra ataques por diccionario, multiplicando exponencialmente el costo de precomputar tablas. PAM permite encadenar varios mecanismos (contraseña, biométricos, multifactor).
- El **control de acceso** se piensa con una matriz recurso $\times$ usuario, pero se implementa como ACLs por columna. UNIX comprime la ACL en 9 bits con las clases *U/G/O*, complementadas con *full ACLs* cuando hace falta más granularidad.
- Hay dos filosofías: **DAC** (el dueño decide; UNIX por defecto) y **MAC** (una autoridad central decide; típico de entornos militares, con modelos como Bell-LaPadula).
- Cada proceso tiene un *real UID* y un **effective UID**. El bit **setuid** permite ejecutar un programa con los privilegios de su dueño, habilitando casos legítimos como **passwd** pero abriendo riesgos de escalado. **su** y **sudo** ofrecen cambios de privilegio bajo control.
- Las **capacidades** reemplazan “setuid-root” por privilegios *finos y específicos*: en lugar de dar todo el poder, dar exactamente lo necesario (por ejemplo, **CAP_NET_RAW** a **tcpdump**).

Idea intuitiva

El principio rector que atraviesa todo el capítulo es el de **mínimo privilegio**: cada usuario, proceso y programa debe tener exactamente los permisos que necesita, ni más ni menos. Las ACLs, **sudo**, las capacidades y los modelos MAC son técnicas distintas para acercarse a ese ideal en escenarios reales.